



TD – Mini Pokédex en MVC PHP



Objectifs pédagogiques

- Comprendre et appliquer le **pattern MVC** (Model – View – Controller).
- Mettre en place un **routeur** dans un fichier séparé.
- Manipuler un **JSON** comme mini base de données.
- Structurer un projet PHP avec POO simple.



Contexte

Vous allez créer un mini **Pokédex** :

- Afficher une **liste de Pokémon** (nom + image).
- Cliquer sur un Pokémon pour voir ses **détails**.
- Les données sont fournies dans un **JSON**, pas besoin de créer ou modifier des données.



Structure du projet à créer

```
/pokedex ou /src
  /controllers
    HomeController.php
    DetailsController.php
  /models
    PokemonModel.php
  /views
    home.php
    details.php
    page404.php // (Bonus)
  /data
    pokemons.json
```

```
index.php
router.php
```

Étape 1 – Préparer le JSON

1. Créez `/data/pokemons.json` .
2. Ajoutez **au moins 3 Pokémon** avec :
 - id
 - name
 - type
 - image (URL)
 - description
3. Testez dans un fichier PHP que vous pouvez **lire le JSON** avec `file_get_contents` et `json_decode` .

💡 Tip : `json_decode($json, true)` retourne un **tableau associatif** en PHP.

Étape 2 – Créer le modèle

1. Créez `PokemonModel.php` dans `/models` .
2. Créez une **classe PokemonModel**.
3. Ajoutez :
 - Une méthode pour **retourner tous les Pokémon** (`getAll`)
 - Une méthode pour **retourner un Pokémon par id** (`getById`)

💡 Tip : Utilisez `foreach` ou `array_filter` pour chercher le Pokémon par id.

💡 Tip pédagogique : le **modèle ne doit jamais afficher quoi que ce soit**, il ne fait que récupérer les données.

Étape 3 – Créer les contrôleurs

1. Créez deux fichiers dans `/controllers` :
 - `HomeController.php`
 - `DetailsController.php`

2. Chaque contrôleur doit être **une classe** simple :

- HomeController → méthode `index()` → récupère tous les Pokémon et les envoie à la vue `home.php` .
- DetailsController → méthode `show($id)` → récupère un Pokémon par id et l'envoie à la vue `details.php` .

💡 Tip : **un contrôleur ne fait pas d'affichage direct** → il prépare juste les données pour la vue.

Étape 4 – Créer le routeur

1. Créez `router.php` .
2. Récupérez la page demandée depuis `$_GET['url']` ou mettez `home` par défaut.
3. Découpez l'URL avec `explode('/')` .
4. Avec un `switch` , **appelez le bon contrôleur** selon la première partie de l'URL :
 - `home` → HomeController
 - `details/:id` → DetailsController avec l'id passé en paramètre

💡 Tip : c'est le **routeur qui décide quel contrôleur exécuter**.

Format d'URL attendu

- Pour afficher la page d'accueil :

```
localhost/index.php?url=home
```

- Pour afficher le détail d'un Pokémon (par exemple id 4) :

```
localhost/index.php?url=details/4
```

⚠ Attention : **on écrit bien** `details/4` **et pas** `details/:4` .

Le chiffre après le `/` est directement l'id du Pokémon.

Étape 5 – Point d'entrée

1. Créez `index.php`.
2. Faites en sorte que **toute la navigation passe par ce fichier**.
3. Incluez le routeur : `require_once 'router.php';`

💡 Tip : `index.php` = **front controller**.

Étape 6 – Créer les vues

1. `/views/home.php` :
 - Affichez la **liste de tous les Pokémon**.
 - Chaque Pokémon doit avoir un **lien vers sa page détail**.
2. `/views/details.php` :
 - Affichez les informations complètes d'un Pokémon : nom, type, image, description.
 - Ajoutez un **lien retour** vers la liste (`home`).

💡 Tip : utilisez du HTML simple et des boucles PHP.

💡 Tip : **ne mettez pas la logique du contrôleur ici**.

Étape 7 – Bonus (optionnel)

- Filtrer les Pokémon par type (`Feu` , `Plante` , etc.) : Pensez à faire une méthode permettant de recenser tous les types contenu dans le `pokemons.json`
 - Mettre en place une erreur 404 : Page non trouvée → Vue par défaut quand le routeur ne comprend pas l'url.
 - Ajouter du CSS pour améliorer le rendu
-

Livrable attendu

- Un mini site MVC fonctionnel :
 - `/home` → liste des Pokémon
 - `/details/:id` → page détail d'un Pokémon
- Respect du **pattern MVC** : modèle, contrôleur, vue séparés.
- Routeur **dans un fichier séparé**.

- Utilisation de **classes pour les contrôleurs**.
 - Présentation finale à l'oral (2-5 min) : expliquer votre structure et le rôle de MVC.
-

Exemple illustré du MVC :

